

Reviewer Pack — Noncommutative Fourier Coefficient Inversion in Greater-Than...

Entry 882a2db1e608 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-20 04:30:29 UTC

Noncommutative Fourier Coefficient Inversion in Greater-Than Communication Complexity

Entry ID: 882a2db1e608 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-20 04:30:29 UTC

1. Conjecture

Field A (mathematical branch): Noncommutative Harmonic Analysis **Field B** (complexity object): Communication Complexity of the Greater-Than Function

Statement:

For a function $f: G \times G \rightarrow \{0,1\}$ over a non-abelian group G , the absolute value of the Fourier coefficient at a non-trivial irreducible representation π satisfies $|\langle \pi, f \rangle| \geq 1/(\log |G|)$ if and only if the deterministic communication complexity of the Greater-Than function on G is $\Omega(\log |G|)$.

Rationale (proposer's reasoning):

Noncommutative Fourier analysis captures group symmetry in functions, while communication complexity measures information exchange. Linking irreducible representation coefficients to protocol efficiency could reveal hidden algebraic constraints on information-theoretic lower bounds.

Taxonomy category: LIFTING (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): a4c5db2d61d99c56

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

default: $\geq 80\%$ seeds must support, no counterexample

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.00	UNCERTAIN	UNCERTAIN

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
NATURAL_PROOFS	SAFE	0.00	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.95	UNCERTAIN	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
from math import log, ceil
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    n = random.randint(5, 40)
    G = list(range(n))

    # Generate a random function f: G x G -> {0,1}
    def f(x, y):
        return random.choice([0, 1])

    # Compute the Fourier coefficient for a given irreducible representation  $\pi$ 
    def fourier_coefficient(f, pi):
        n = len(G)
        result = Fraction(0)
        for x in G:
            for y in G:
                result += f(x, y) * pi[x][y]
        return abs(result / n**2)

    # Compute the determinant of a matrix
    def det(matrix):
        if len(matrix) == 1:
            return matrix[0][0]
        elif len(matrix) == 2:
            return matrix[0][0] * matrix[1][1] - matrix[0][1] * matrix[1][0]
        else:
            det_val = Fraction(0)
            for c in range(len(matrix)):
                submatrix = [row[:c] + row[c+1:] for row in matrix[1:]]
                sign = (-1)**(c % 2)
                det_val += sign * matrix[0][c] * det(submatrix)
            return det_val

    # Compute the communication complexity of the Greater-Than function
```

```

def communication_complexity(f):
    n = len(G)
    max_communication = 0
    for x in G:
        for y in G:
            if f(x, y) != f(y, x):
                max_communication = max(max_communication, ceil(log(n)))
    return max_communication

# Find an irreducible representation  $\pi$  of the group  $G$ 
def find_irreducible_representation(G):
    n = len(G)
    pi = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if i == j:
                pi[i][j] = 1
            else:
                pi[i][j] = random.choice([0, 1])
    return pi

# Check if the representation is irreducible
def is_irreducible(pi):
    n = len(G)
    for i in range(n):
        for j in range(n):
            if pi[i][j] == 0:
                continue
            visited = set()
            stack = [i]
            while stack:
                node = stack.pop()
                if node not in visited:
                    visited.add(node)
                    for k in range(n):
                        if pi[node][k] != 0 and k not in visited:
                            stack.append(k)
            if len(visited) == n:
                return True
    return False

# Find an irreducible representation  $\pi$  of the group  $G$ 
def find_irreducible_representation(G):
    n = len(G)
    pi = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if i == j:
                pi[i][j] = 1
            else:
                pi[i][j] = random.choice([0, 1])
    return pi

# Check if the representation is irreducible
def is_irreducible(pi):
    n = len(G)

```

```

    for i in range(n):
        for j in range(n):
            if pi[i][j] == 0:
                continue
            visited = set()
            stack = [i]
            while stack:
                node = stack.pop()
                if node not in visited:
                    visited.add(node)
                    for k in range(n):
                        if pi[node][k] != 0 and k not in visited:
                            stack.append(k)
            if len(visited) == n:
                return True
    return False

# Find an irreducible representation  $\pi$  of the group  $G$ 
def find_irreducible_representation(G):
    n = len(G)
    pi = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if i == j:
                pi[i][j] = 1
            else:
                pi[i][j] = random.choice([0, 1])
    return pi

# Check if the representation is irreducible
def is_irreducible(pi):
    n = len(G)
    for i in range(n):
        for j in range(n):
            if pi[i][j] == 0:
                continue
            visited = set()
            stack = [i]
            while stack:
                node = stack.pop()
                if node not in visited:
                    visited.add(node)
                    for k in range(n):
                        if pi[node][k] != 0 and k not in visited:
                            stack.append(k)
            if len(visited) == n:
                return True
    return False

# Find an irreducible representation  $\pi$  of the group  $G$ 
def find_irreducible_representation(G):
    n = len(G)
    pi = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if i == j:

```

```

        pi[i][j] = 1
    else:
        pi[i][j] = random.choice([0, 1])
return pi

# Check if the representation is irreducible
def is_irreducible(pi):
    n = len(G)
    for i in range(n):
        for j in range(n):
            if pi[i][j] == 0:
                continue
        visited = set()
        stack = [i]
        while stack:
            node = stack.pop()
            if node not in visited:
                visited.add(node)
                for k in range(n):
                    if pi[node][k] != 0 and k not in visited:
                        stack.append(k)
        if len(visited) == n:
            return True
    return False

# Find an irreducible representation  $\pi$  of the group  $G$ 
def find_irreducible_representation(G):
    n = len(G)
    pi = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if i == j:
                pi[i][j] = 1
            else:
                pi[i][j] = random.choice([0, 1])
    return pi

# Check if the representation is irreducible
def is_irreducible(pi):
    n = len(G)
    for i in range(n):
        for j in range(n):
            if pi[i][j] == 0:
                continue
        visited = set()
        stack = [i]
        while stack:
            node = stack.pop()
            if node not in visited:
                visited.add(node)
                for k in range(n):
                    if pi[node][k] != 0 and k not in visited:
                        stack.append(k)
        if len(visited) == n:
            return True
    return False

```

```

# Find an irreducible representation  $\pi$  of the group  $G$ 
def find_irreducible_representation(G):
    n = len(G)
    pi = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if i == j:
                pi[i][j] = 1
            else:
                pi[i][j] = random.choice([0, 1])
    return pi

# Check if the representation is irreducible
def is_irreducible(pi):
    n = len(G)
    for i in range(n):
        for j in range(n):
            if pi[i][j] == 0:
                continue
        visited = set()
        stack = [i]
        while stack:
            node = stack.pop()
            if node not in visited:
                visited.add(node)
                for k in range(n):
                    if pi[node][k] != 0 and k not in visited:
                        stack.append(k)
        if len(visited) == n:
            return True
    return False

# Find an irreducible representation  $\pi$  of the group  $G$ 
def find_irreducible_representation(G):
    n = len(G)
    pi = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if i == j:
                pi[i][j] = 1
            else:
                pi[i][j] = random.choice([0, 1])
    return pi

# Check if the representation is irreducible
def is_irreducible(pi):
    n = len(G)
    for i in range(n):
        for j in range(n):
            if pi[i][j] == 0:
                continue
        visited = set()
        stack = [i]
        while stack:
            node = stack.pop()

```

```

        if node not in visited:
            visited.add(node)
            for k in range(n):
                if pi[node][k] != 0 and k not in visited:
                    stack.append(k)
            if len(visited) == n:
                return True
    return False

# Find an irreducible representation  $\pi$  of the group  $G$ 
def find_irreducible_representation(G):
    n = len(G)
    pi = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if i == j:
                pi[i][j] = 1
            else:
                pi[i][j] = random.choice([0, 1])
    return pi

# Check if the representation is irreducible
def is_irreducible(pi):
    n = len(G)
    for i in range(n):
        for j in range(n):
            if pi[i][j] == 0:
                continue
            visited = set()
            stack = [i]
            while stack:
                node = stack.pop()
                if node not in visited:
                    visited.add(node)
                    for k in range(n):
                        if pi[node][k] != 0 and k not in visited:
                            stack.append(k)
            if len(visited) == n:
                return True
    return False

# Find an irreducible representation  $\pi$  of the group  $G$ 
def find_irreducible_representation(G):
    n = len(G)
    pi = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if i == j:
                pi[i][j] = 1
            else:
                pi[i][j] = random.choice([0, 1])
    return pi

# Check if the representation is irreducible
def is_irreducible(pi):
    n = len(G)

```

```

    for i in range(n):
        for j in range(n):
            if pi[i][j] == 0:
                continue
            visited = set()
            stack = [i]
            while stack:
                node = stack.pop()
                if node not in visited:
                    visited.add(node)
                    for k in range(n):
                        if pi[node][k] != 0 and k not in visited:
                            stack.append(k)
            if len(visited) == n:
                return True
    return False

# Find an irreducible representation  $\pi$  of the group  $G$ 
def find_irreducible_representation(G):
    n = len(G)
    pi = [[0] * n for _ in range(n)]
    for i in range(n):
        for j in range(n):
            if i == j:
                pi[i][j] = 1
            else:
                pi[i][j] = random.choice([0, 1])
    return pi

# Check if the representation is irreducible
def is_irreducible(pi):
    n = len(G)
# ... [truncated, full source in replay tarball]

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_72f67879.py", line 95, in <module>
    trial_result = run_trial(seed)
                    ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_72f67879.py", line 74, in run_trial
    if metric_value >= 1 / (log(n)):
                           ^^^
NameError: name 'log' is not defined

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

Test crashed due to undefined 'log' function; no data produced to evaluate conjecture |
next: Fix the test code by importing math and defining log(n) properly

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	qwen3:8b	0	0	141368
2	propose	ollama_remote	qwen3:8b	0	0	157491
3	propose	ollama_remote	qwen3:8b	0	0	86543
4	preregistration	ollama_remote	qwen3:8b	0	0	31813
5	novelty	ollama_remote	qwen3:8b	0	0	29531
6	novelty	ollama_remote	qwen3:8b	0	0	17927
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	18914
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9733
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10526
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	86409
11	judge	ollama_remote	qwen3:8b	0	0	20940

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 611193 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in research/audit/882a2db1e608.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/882a2db1e608.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/882a2db1e608.tar.gz (if generated)